

Obtain protein annotations

Tom Smith

2026-08-24

Contents

1	Purpose	1
2	Setup	2
3	Collect protein accessions	2
3.1	Uniprot protein details	2
3.2	GO terms	3

1 Purpose

This notebook fetches UniProt annotations (gene names, protein names, and GO terms) for every protein detected in either of the two MaxQuant searches (the PER2-HALO immunoprecipitation experiment and the PER2-PROTAC experiment) and saves them to `external/` for use by all subsequent notebooks. Running it separately from the quantification pipeline means annotations only need to be re-fetched if the set of detected proteins changes, not every time the raw quantification data is re-processed.

Input: `raw/*/evidence.txt*` — the evidence (PSM-level) output from MaxQuant for both the HALO and PROTAC searches. Only the `Leading.razor.protein` column is used here.

Output:

- `external/uniprot2details_unique.rds` — UniProt metadata (entry name, protein names, gene names, organism, sequence length, etc.), one row per UniProt accession, collapsed to a single row where an accession returned multiple rows from UniProt. This is the object joined onto the quantification data in notebooks 1-4.
- `external/go_res_all.rds` — Gene Ontology (GO) terms for each protein, expanded to include ancestor (parent) terms. This is not currently read by any other notebook in this repository; it is produced here, alongside the protein details, so that GO enrichment analyses can be added later without a second pass over the raw data.

Analysis:

1. **Collect protein accessions** — read the `Leading.razor.protein` column from every `evidence.txt` file under `raw/` (both experiments), splitting any protein-group strings (multiple accessions separated by `;`) into individual accessions, to get the full set of UniProt IDs detected anywhere in the project.

2. **UniProt protein details** — query the UniProt REST API (via `uniprotREST::uniprot_map`) for each accession to retrieve gene names, protein names, and other metadata. Accessions that return multiple rows (because the accession has since been “demerged” into separate UniProt entries) are collapsed to a single row per accession by concatenating the values.
3. **GO terms** — separately query the UniProt API for the GO terms of each accession, reshape the result to one row per protein-term pair, and expand the annotation set to include every ancestor term in the GO hierarchy, not just the directly-annotated terms.

Note: steps 2 and 3 contact the UniProt REST API and may take several minutes, depending on the number of proteins and UniProt server load. The saved RDS files act as a cache; re-running this notebook is only necessary if the set of detected proteins changes (e.g. after a raw data re-search), and since UniProt entries and GO annotations are updated over time, the exact annotation values may differ slightly between runs performed months apart.

2 Setup

3 Collect protein accessions

We locate every `evidence.txt` file under `raw/` (one per MaxQuant search: HALO and PROTAC) and pool the `Leading.razor.protein` values across both, since a protein detected in either experiment needs annotating. MaxQuant occasionally reports several accessions for one feature as a single `;`-separated protein-group string when the peptides cannot distinguish between them; we split these apart so that each individual accession is queried against UniProt.

```
maxQuant_outfiles <- Sys.glob(here('raw/*/evidence.txt*'))
```

```
all_proteins <- maxQuant_outfiles %>% lapply(function(x){
  read.delim(x) %>%
  pull(Leading.razor.protein) %>%
  unique() %>%
  strsplit(';') %>%
  unlist() %>% unique()
}) %>% unlist() %>% unique()

length(all_proteins)
```

```
## [1] 9612
```

3.1 Uniprot protein details

Below, we obtain the protein details from UniProt using the `uniprotREST` package, which wraps the UniProt REST API. We rename the `From` column (the accession we queried with) to `UniprotID`, drop the redundant `Entry` column, and derive `Gene.Names.First` — the first gene name listed in UniProt’s (space-separated) `Gene.Names` field, which we treat as the canonical gene symbol for plotting and labelling throughout the rest of the analysis.

```
uniprot2details <- uniprot_map(
  all_proteins,
  from='UniProtKB_AC-ID',
```

```

    method='stream') %>%
  dplyr::rename('UniprotID'=From) %>%
  select(-Entry) %>%
  mutate(Gene.Names.First=gsub(' .*', '', Gene.Names))

## Running job: f0lzF2yOXf
## Checking job status...

## Job complete!
## Downloading: 0 B      Downloading: 0 B      Downloading: 0 B      Downloading: 0 B      Downloading: 0 B

# Some UniprotIDs generate multiple rows. It looks like this is because they have been subsequently 'de
uniprot2details_unique <- uniprot2details %>%
  group_by(UniprotID) %>%
  summarise_all(.funs=function(x) paste(x, collapse=';'))

saveRDS(uniprot2details_unique, here('external/uniprot2details_unique.rds'))

```

3.2 GO terms

Below we obtain the GO terms from UniProt using the API and then we reformat the output before expanding the GO terms to include all parent terms too. GO over-representation testing packages typically only consider the terms directly annotated to a protein, not the (implied) parent terms further up the GO hierarchy; expanding the annotation set here means downstream enrichment analyses will not need to repeat this step.

```

tempfilename <- tempfile()

uniprot_map(
  ids = all_proteins,
  from = "UniProtKB_AC-ID",
  method='stream',
  to = "UniProtKB", compressed = TRUE,
  format = "tsv", path = tempfilename,
  fields = "go",
)

## Running job: f0lzF2yOXf
## Checking job status...

## Job complete!
## Downloading: 0 B      Downloading: 0 B      Downloading: 0 B      Downloading: 0 B      Downloading: 0 B

go_res <- read.delim(tempfilename) %>% dplyr::rename(c('UNIPROTKB'='From'))

file.remove(tempfilename)

## [1] TRUE

```

```

# The Uniprot API returns a data.frame with all the GO terms for each protein in a single column
# Below, we separate each GO terms to be on a different row
go_res <- go_res %>%
  filter(Gene.Ontology..GO.!='') %>% separate_rows(Gene.Ontology..GO., sep = '; ') %>%
  separate_wider_delim(Gene.Ontology..GO., ' [GO:', names=c('GO_desc', 'GO.ID')) %>%
  mutate(GO.ID=paste0('GO:', gsub(']$', '', GO.ID)))

# And then expand the GO terms so that all ancestor/parent terms are annotated to each protein
# This is to avoid issues with GO over-representation testing packages only using
# the directly annotated terms and not the parent terms
go_res_all <- get_ancestor_go(go_res)

```

```
## [1] "Getting all ancestor GO terms for 13822 observed terms. This may take a while!"
```

Save to file for use in other notebooks.

```
saveRDS(go_res_all, here('external/go_res_all.rds'))
```